Comparative performance analysis between ResNet18 and ResNet34 architectures for CIFAR-10

Andrea Colombo - Artificial Intelligence course project - A.A 2025/2026

Abstract — This report contains an integrative analysis for the first report of the final project of the artificial intelligence course. In this document we explore the ResNet34 architecture and how it performs on CIFAR-10 compared to the previously implemented ResNet18 network. The goal is to conduct a comparative analysis to unveil whether a deeper network can yield significant performance gains for this particular use case.

Table of Contents

1 Introduction 2
1.1 Motivation and Objectives 2
2 Network Implementation 2
3 Training a Deeper Network 3
3.1 Same techniques, different networks 3
3.1.1 Training performance 3
3.1.2 Test Performance 3
3.1.3 Performance Comparison 4
3.2 Modified Training Approach 4
3.2.1 Enhanced Data Augmentation 4
3.2.2 Learning Rate Scheduling 5
3.2.3 Stronger Weight Decay 5
3.2.4 Increasing Early Stopping patience 5
3.2.5 Training Performance 5
3.2.6 Results on Test Dataset 6
3.2.7 Results Analysis 6
3.2.8 Per Class Performance 6
4 Comparative Analysis 8
4.1 Training Performance 8
4.2 Performance on Test Dataset 8
4.3 Results Analysis 8
4.4 Per Class Performance 9
5 Conclusions 10
5.1 Answering the Initial Questions 10
Bibliography 11

1 Introduction

As stated in the abstract, this file contains an integrative analysis on the ResNet34 architecture [1], a deeper residual network compared to the previously implemented and analyzed ResNet18. This document will not cover the introductory part about CIFAR-10 [2] and residual networks covered in the previous report but it focuses more on the network implementation and how the additional layers influence both the training process and the results.

1.1 Motivation and Objectives

Through this analysis we are going to cover some key points that will give us more insight about deeper networks.

- Does increasing the network depth lead to higher accuracy on CIFAR-10?
- What is the tradeoff between the gained accuracy and computational cost?
- How does the increased number of layers affect overfitting in residual networks?

2 Network Implementation

The network implementation itself is not extremely different from the previously described ResNet18. Since the residual block implementation hasn't changed from the previous report, only the ResNet34 class is displayed in the following code block.

```python
class ResNet34(nn.Module):
  def __init__(self):
      super(ResNet34, self).__init__()
      self.conv_1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
      self.bnorm_1 = nn.BatchNorm2d(64)
      self.res_1 = self.make_blocks(64,  64,  3, 1)
      self.res_2 = self.make_blocks(64,  128, 4, 2)
      self.res_3 = self.make_blocks(128, 256, 6, 2)
      self.res_4 = self.make_blocks(256, 512, 3, 2)
      self.linear = nn.Linear(512, 10)

  def make_blocks(self, in_chs, out_chs, nblocks, stride):
      layers = []
      layers.append(ResBlock(in_chs, out_chs, stride))
      for _ in range(1, nblocks):
          layers.append(ResBlock(out_chs, out_chs, stride=1))
      return nn.Sequential(*layers)

  def forward(self, x):
      out = F.relu(self.bnorm_1(self.conv_1(x)))
      out = self.res_1(out)
      out = self.res_2(out)
      out = self.res_3(out)
      out = self.res_4(out)
      out = F.avg_pool2d(out, 4)
      out = out.view(out.size(0), -1)
      out = self.linear(out)
      return out
```

3 Training a Deeper Network

3.1 Same techniques, different networks

The first experiment conducted with this architecture was to train it using the same hyperparameters and techniques used in the ResNet18 implementation to see if there's any meaningful difference in performance.

The table below reports the hyperparameter values used for this training experiment.

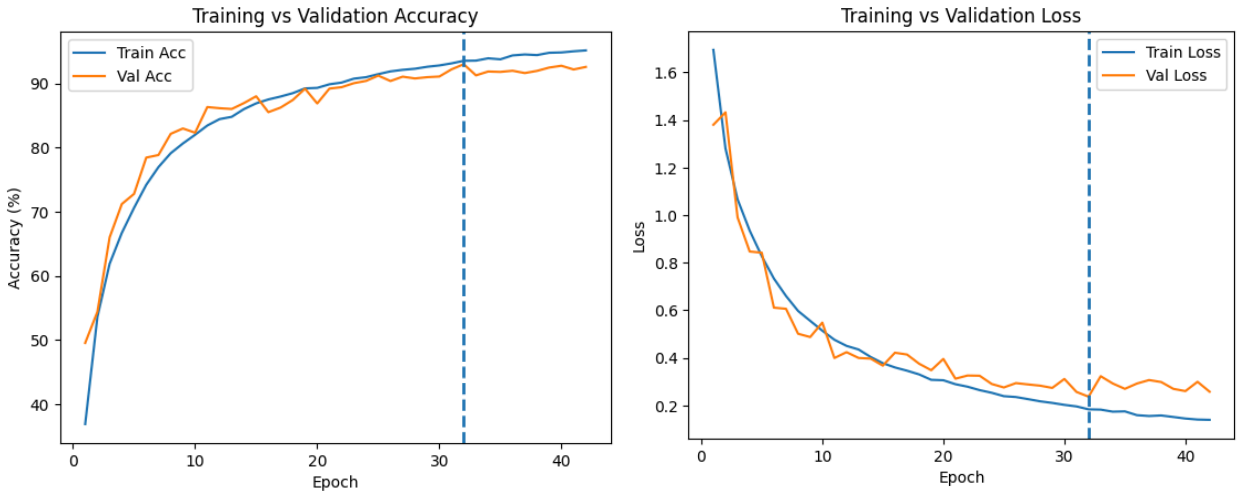
Parameter	Value
max epochs	100
batch size	128
learning rate	1e-3
weight decay	1e-4
patience	10
min delta	1e-3

Table 1: Hyperparameter values (same as the ResNet18 implementation)

3.1.1 Training performance

The network was trained for 42 epochs before the early stopping was triggered, with the best model saved at epoch 32.

The plots below display the accuracy and loss curves for the training and validation sets for this training run.



3.1.2 Test Performance

The model achieved 91.96% accuracy on the test set, showing no significant advantage compared to the ResNet18 architecture.

The ResNet18 described in the previous report achieved 92.38% accuracy.

3.1.3 Performance Comparison

As seen in the previous sections, training a deeper architecture with the same approach as the one used for the ResNet18 doesn't guarantee better results.
The table below showcases the differences for training and test metrics between the two implemented networks.

Metric	ResNet18	ResNet34	Difference
Test Accuracy	92.38%	91.96%	−0.42%
Best Validation accuracy	93.04%	93.00%	−0.04%
Best Epoch	44	32	**−27%**
Total Epochs	54	42	**−22%**
Trainable Parameters	11.17M	21.28M	**+90%**
Training Accuracy	95.14%	93.53%	−1.51%
Overfitting Gap	2.76%	1.57%	**−43%**

Table 2: Performance comparison between ResNet18 and ResNet34

Despite having 90% more trainable parameters, the ResNet34 reached its peak in validation accuracy 27% faster compared to the ResNet18. This faster convergence suggests that the additional layers enabled more efficient feature learning in the early training stages.

The training curves followed a very similar pattern for both models with a very rapid learning in the early stages followed by gradual refinement.

3.2 Modified Training Approach

The similar performance seen in Section 3.1 suggests that the deeper network may require different training strategies to fully take advantage of its potential.
In this section we explore some possible improvements to the training process and how they affect the performance of the network.

3.2.1 Enhanced Data Augmentation

As we were able to see in the ResNet18 implementation, data augmentation is an effective technique when it comes to improving the model's accuracy. Traditional data augmentation relies on fixed transformations (e.g. rotations and cropping) with manually chosen probabilities and magnitudes. AutoAugment [3] uses reinforcement learning to search for the best augmentation policies in a discrete search space.

The key advantages of AutoAugment are:

- **Task specific optimizations**: augmentation policies are learned for each dataset.
- **Operation sequencing**: AutoAugment discovers the most beneficial combinations of transformations.
- **Automatic magnitude tuning**: the optimal intensity for each operation is chosen automatically.

Training AutoAugment from scratch requires thousands of GPU hours, making it unfeasible for this project. The proposed implementation uses PyTorch's built-in AutoAugment with pre-learned CIFAR-10 policies.

3.2.2 Learning Rate Scheduling

The constant learning rate used for the baseline test (1e-3) might be suboptimal for the ResNet34's deeper architecture. Deeper networks often benefit from gradually reducing the learning rate during training. This enables a more aggressive learning phase at the beginning followed by a fine tuning phase in the later epochs.

This implementation used cosine annealing, smoothly decreasing the learning rate from the initial value of *1e-3* to the final value *1e-6* following a cosine curve over the training period.

3.2.3 Stronger Weight Decay

The previous weight decay value (1e-4) was chosen for the ResNet18 architecture and its 11.17M parameters. With the ResNet34 and the 90% increase in trainable parameters we may need a higher value to prevent overfitting and ensure better generalization.
The value was therefore increased to 5e-4 with the goal of providing more aggressive L2 regularization.

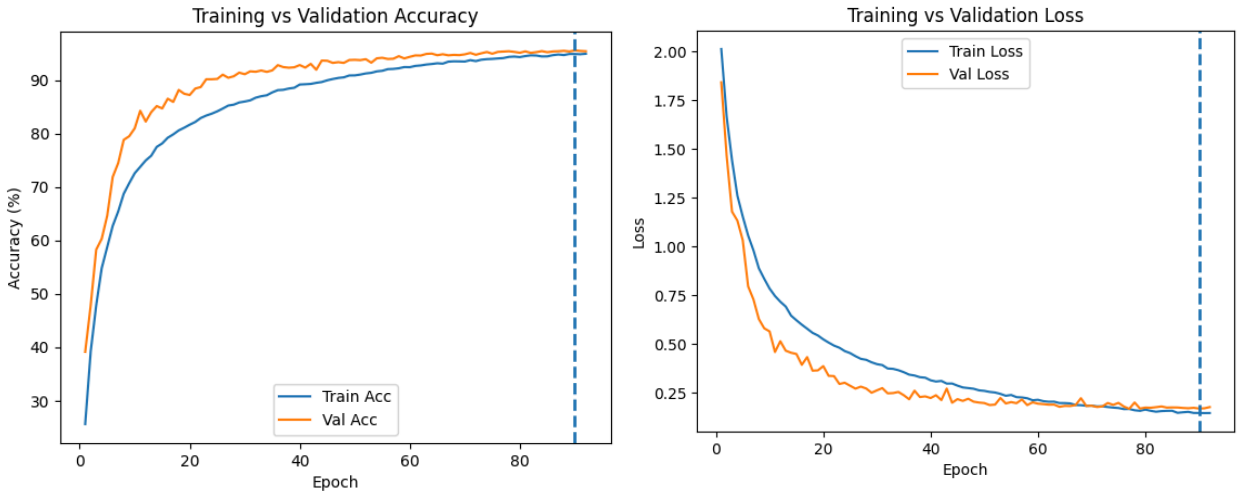
3.2.4 Increasing Early Stopping patience

In order to experiment more with the changes described above, the early stopping patience was increased from 10 to 15 epochs.

3.2.5 Training Performance

The model was trained for 92 epochs before the early stopping was triggered, reaching a peak of 95.54% in validation accuracy at epoch 90.

The following plots show the validation and loss curves for the training and validation datasets.



As we can see from the accuracy curves, the model performed better on the validation dataset for the entirety of the training. This shows that the more aggressive data augmentation approach successfully regularized the model, making the augmented training data harder to learn compared to the untouched validation data.

3.2.6 Results on Test Dataset

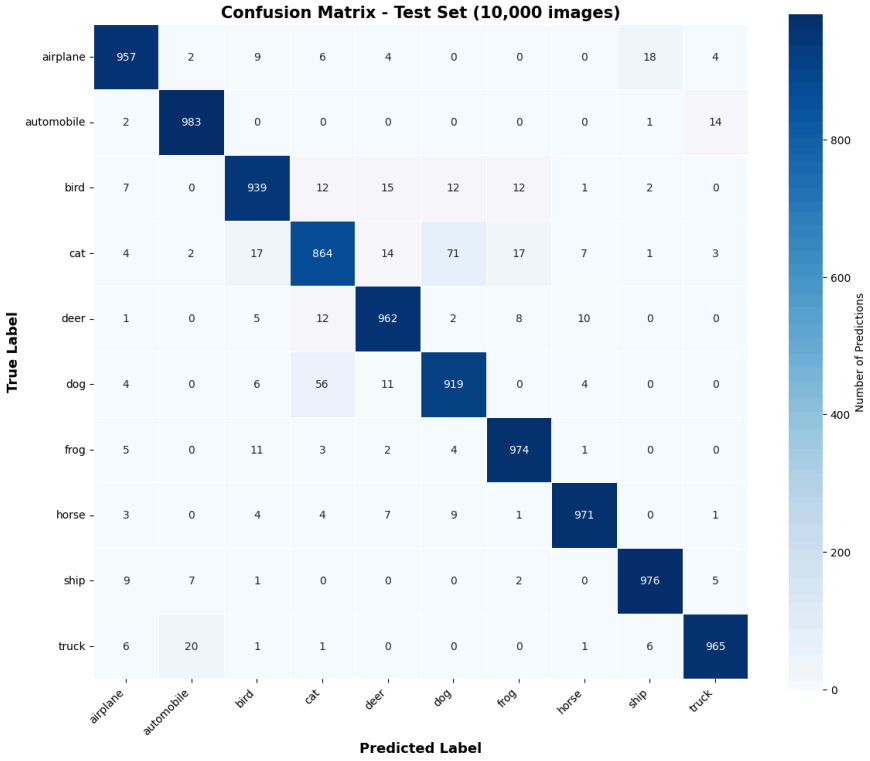
The model trained with the methodologies described above achieved **95.1%** accuracy on the test set, showing a meaningful improvement over both the ResNet18 implementation and the baseline ResNet34 implementation described in the early sections of this document.

3.2.7 Results Analysis

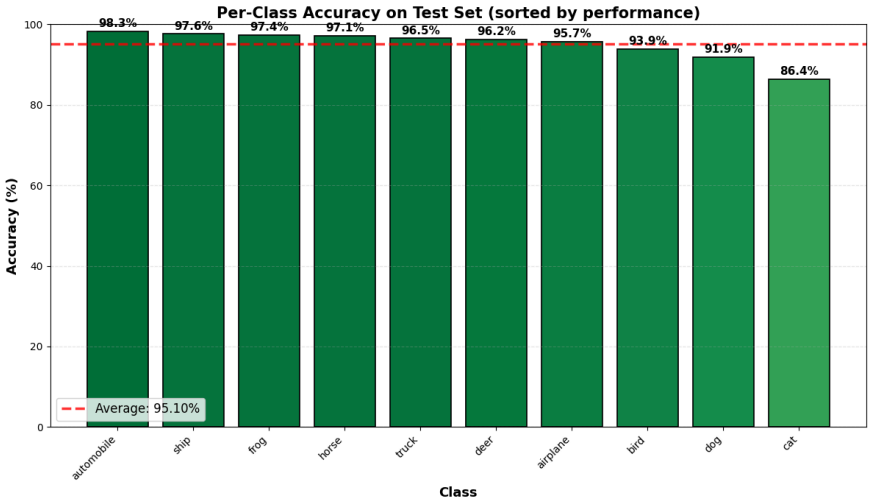
By comparing this result to the one obtained in Section 3.1.3, we can see how the improvements made to the training procedure led to bigger performance gains than just switching to a deeper network.

Thanks to the learning rate scheduling and bigger early stopping patience, the network was able to keep training for a much longer time period compared to the baseline implementation, these changes played a crucial role in the later stages of the training. The more aggressive data augmentation pipeline forced the network to learn more generalizable features rather than surface level patterns. This can be seen in the training plots, the validation accuracy has always been higher than the training accuracy, this means that the combination of the previous data augmentation techniques with AutoAugment made the training set much harder to train for the network, leading to the -0.19% gap between test accuracy and training accuracy.

3.2.8 Per Class Performance



The confusion matrix above reveals that the most frequently confused classes did not change with respect to the baseline ResNet18 implementation, with cats and dogs being the most confused classes followed by birds and planes. This was expected because the confusions do not stem from architectural flaws but rather from the characteristics of the dataset itself.



By examining the plot above, it is observable how the network shows strong accuracy across all classes, with most of them exceeding 93% accuracy. This reveals that the changes applied to the training approach improved the network performance in a uniform way across all classes without benefitting a subset of them in a disproportionate way. Although the most challenging classes remained the same, we were able to get a strong improvement over the baseline ResNet18 implementation. This suggests that, even if a deeper architecture and a better training approach can yield solid performance gains, the underlying challenges of the dataset remain the same.

4 Comparative Analysis

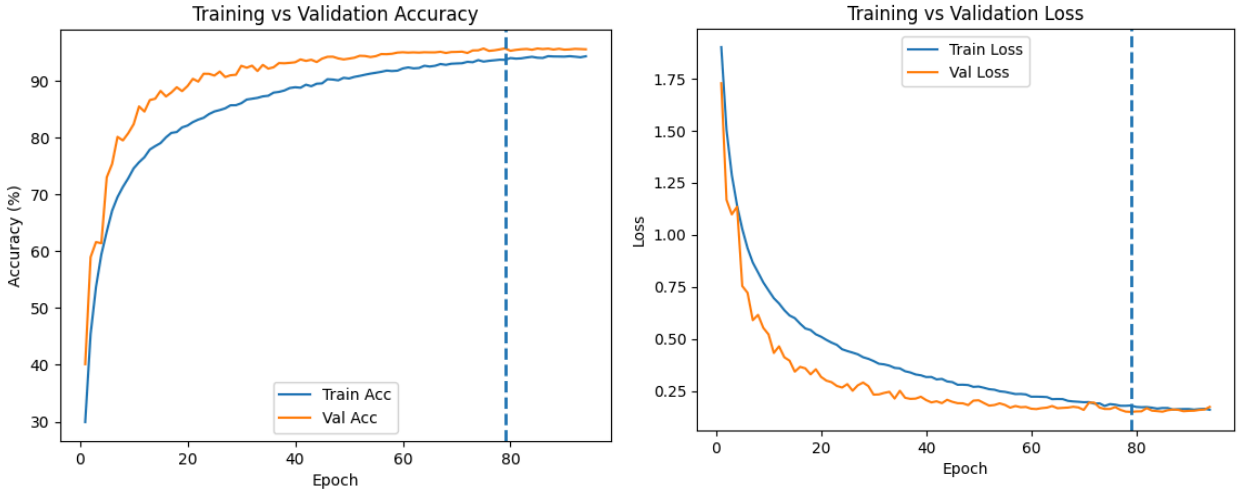
In order to distinguish the performance gains stemmed from the deeper network architecture from the ones generated by the improvements made to the training approach, we conducted a comparative analysis by training the previously described ResNet18 with the new training configuration Section 3.2.

4.1 Training Performance

The network was trained for 94 epochs before the early stopping was triggered, with the validation accuracy reaching its peak of 95.72% at epoch 72.

The modifications applied to the training approach resulted in a 70% increase in training time compared to the baseline ResNet18 implementation described in the previous report.

This result matches the one seen in the ResNet34 training, showing that the two networks exhibited nearly identical behavior during training.



As we saw in the improved ResNet34, the model performed better on the validation set. This is caused by the aggressive data augmentation pipeline chosen for this use case.

4.2 Performance on Test Dataset

The model obtained 94.61% accuracy on the test dataset, showing a significant improvement over the baseline implementation proposed in the last report and almost matching the accuracy obtained by the deeper ResNet34.

4.3 Results Analysis

The optimized ResNet18 achieved 94.61% test accuracy compared to 95.1% for the optimized ResNet34.

This is the central finding of this report: while ResNet34 requires 90% more parameters and approximately twice the training time, its advantage over a properly trained ResNet18 shrinks to less than half a percentage point.

Model	Training	Test Accuracy	Gain over ResNet18 baseline
ResNet18	Baseline	92.38%	—
ResNet34	Baseline	91.96%	−0.42%
ResNet18	Optimized	94.61%	+2.23%
ResNet34	Optimized	95.1%	+2.72%

Table 3: Full performance comparison across training configurations

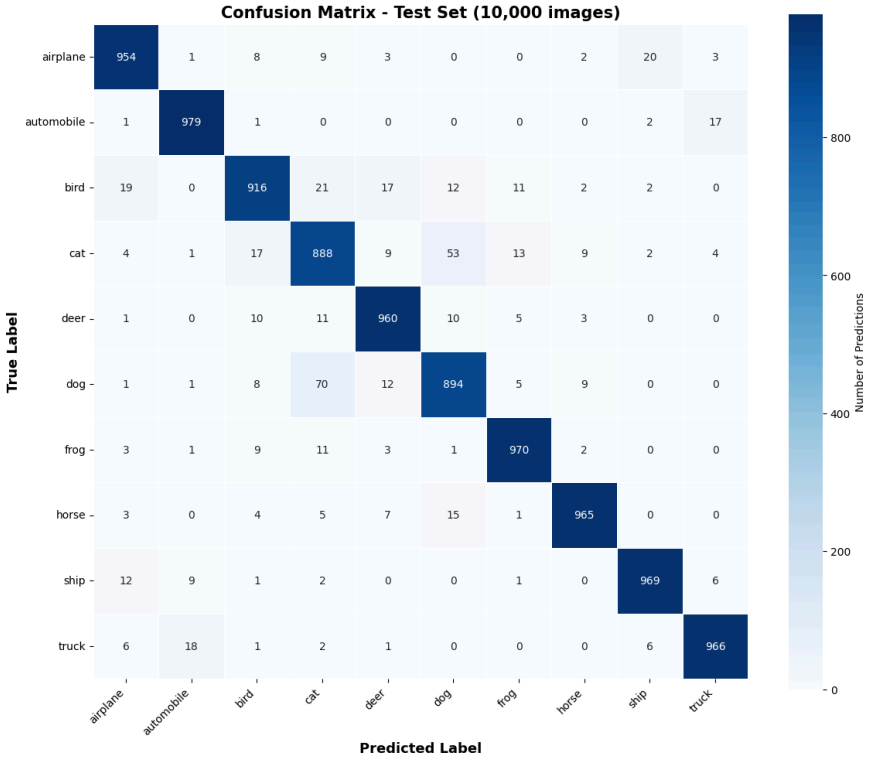
The improvement achieved by the optimized ResNet34 over the baseline can be approximately decomposed as follows:

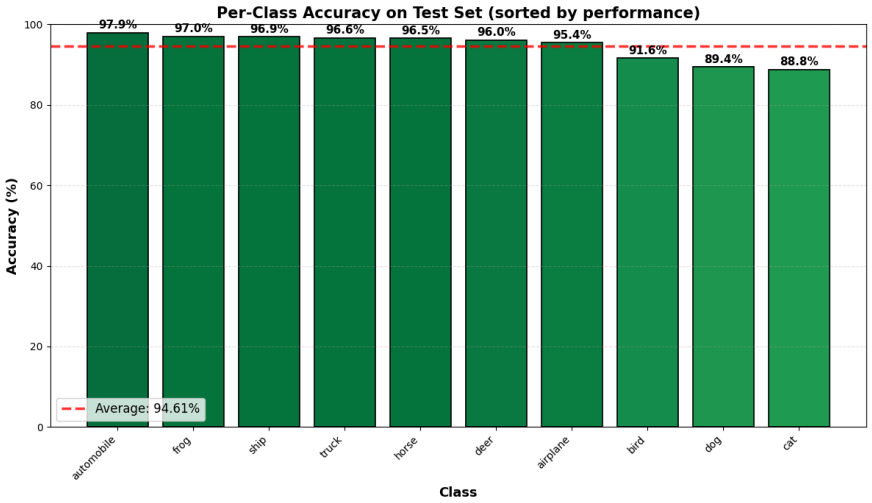
- around 2.23% came from better training techniques alone
- only 0.49% can be attributed to the additional depth of ResNet34.

In other words, roughly 85% of the performance gain came from how the network was trained, and only 15% from which network was used.

4.4 Per Class Performance

The per class performance plots are included for completeness but, as we have seen in the earlier sections, the different training had no influence on them.





As we expected, the expressed behavior on the different classes is almost identical to the one seen on the baseline ResNet18 and the ResNet34 implementations.

5 Conclusions

The comparative analysis reported in this document showed that, for the specific use case of CIFAR-10, using a deeper network does not guarantee significant performance gains.
The goal of this analysis was not to determine which architecture is better in a general sense, what we wanted to unveil was if, for this very specific use case, using a ResNet34 would lead to better performance. Both architectures achieved competitive accuracy levels and both can be used depending on the constraint of the project.

- A ResNet18 would be a great fit for resource constrained environments.
- A ResNet34 could be a good fit for projects where the benefit of the extra percentage points in accuracy outweighs the price of the additional computation.

For small to medium scale datasets like CIFAR-10, the training methodology and regularization techniques matter more than just the sheer amount of layers in the network.

5.1 Answering the Initial Questions

Does increasing the network depth lead to higher accuracy on CIFAR-10?

As seen throughout this comparative analysis, just switching to a deeper network does not guarantee better results. If we look at the baseline ResNet34 implementation that used the same training approach as the ResNet18 it's shown how there was no performance gain at all.
However, by adapting the training methodology to the deeper architecture, we were able to extract more performance from it.

What is the tradeoff between the gained accuracy and computational cost?

The baseline comparison showed no meaningful tradeoff — ResNet34 required 90% more parameters and converged in fewer epochs, yet delivered lower test accuracy (91.96% vs 92.38%). Only with optimized training did ResNet34 justify its increased cost, achieving 95.1% versus ResNet18′s baseline of 92.38%.

However, when comparing both architectures under optimized training, the tradeoff becomes less favorable for ResNet34. The shallower ResNet18 reached 94.61% accuracy with approximately half the parameters and shorter total training time. The remaining 0.49% advantage of ResNet34 must be weighed against its 90% parameter overhead and longer training requirement. On higher-resolution datasets such as ImageNet, where spatial complexity benefits deeper feature hierarchies, this tradeoff would likely shift further in ResNet34′s favor.

How does the increased number of layers affect overfitting in residual networks?

The baseline results showed that the deeper ResNet34 actually exhibited less overfitting than ResNet18 despite having 90% more parameters (1.57% gap vs 2.76%), suggesting that the residual connections in deeper networks provide implicit regularization by allowing gradient flow through identity shortcuts.
With the optimized training, this trend continued to a remarkable degree: the optimized ResNet34 exhibited a negative overfitting gap of -0.19%, meaning test accuracy exceeded training accuracy. The optimized ResNet18 showed similar behavior. This outcome, unusual in deep learning, is a direct consequence of AutoAugment's aggressive data augmentation making the training set harder to classify than the clean test set. It demonstrates that with appropriate regularization, increased network depth does not lead to worse overfitting, in fact, the combination of depth and strong regularization can produce exceptionally well generalizing models.

Bibliography

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.

[2] A. Krizhevsky and G. Hinton, "Learning Multiple Layers of Features from Tiny Images," technical report, 2009.

[3] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le, "AutoAugment: Learning augmentation strategies from data," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 113–123.